Reward Shaping for Multi-Objective Survival in Grid-Based Deep Reinforcement Learning

Philipp Merz

2026

Abstract

We study how reward function design affects learned behavior in a multi-objective survival task. An agent on a 64×64 grid must eat food to avoid starvation, evade enemies, and use shelters for protection. We train a Rainbow-lite DQN agent under several reward configurations and compare against a scripted heuristic baseline. We document three failure modes that arise from reward weight imbalances and derive seven feasibility constraints that prevent them. A minimal reward function with four components outperforms a nine-component alternative, but no reward configuration matches the heuristic: the agent consistently learns to shelter-camp rather than forage, and scaling the food reward does not resolve this. Adding temporal context through frame stacking, without changing the reward function, produces an agent that significantly outperforms the heuristic (811 vs 768 mean survival, $p = 0.007$). We propose three hypotheses for why reward shaping alone fails and suggest falsification experiments for each.

Contents

1 Introduction **3**

2 Background **3**
2.1 Reinforcement Learning Fundamentals . . . 3
2.2 Deep Q-Networks . . . 4
2.3 DQN Extensions . . . 4
2.4 Training Stability Techniques . . . 5
2.5 Reward Shaping . . . 6

3 Environment **6**
3.1 World Design . . . 6
3.2 Observation Space . . . 7
3.3 Action Space . . . 7
3.4 Reward Structure . . . 8

4 Agent Architecture **9**
4.1 Network . . . 9
4.2 Training Algorithm . . . 10
4.3 Hyperparameters . . . 11

5 Experiments **11**
5.1 Experimental Setup 11
5.2 Failure Modes 12
5.3 Reward Configurations 12
5.4 Engineered Reward Progression 14
5.5 Temporal Context 15
5.6 Results 15
5.7 Summary of Observations 16

6 Discussion **16**

7 Conclusion **18**

1 Introduction

Reinforcement learning (RL) has produced agents that match or exceed human performance on complex tasks, from Atari games [Mnih et al., 2015] to board games and robotic control. These results typically require large-scale compute. A less studied but equally important problem is the design of the reward function itself.

Ng et al. [1999] proved that potential-based reward shaping preserves the optimal policy. However, in practice, reward functions for complex tasks are rarely potential-based. They combine multiple event-driven components (bonuses for collecting resources, penalties for taking damage, terminal rewards for death) whose relative weighting directly determines what the agent optimizes. Small changes in these weights can produce qualitatively different behaviors.

Multi-objective environments amplify this problem. A survival task requires balancing resource gathering, which demands exploration, against threat avoidance, which favors conservative behavior. The reward function must encode the relative importance of these objectives. There is no single "correct" weighting; the choice reflects a design decision about what behavior is desirable.

Most reward shaping research evaluates on single-objective benchmarks (Atari, MuJoCo locomotion) where the reward is given or straightforward to define. Practitioners building agents for multi-objective tasks with competing pressures lack principled guidance: which components to include, how to set their relative weights, and when reward shaping itself is insufficient. This work addresses that gap. We derive feasibility constraints that rule out degenerate reward configurations, show empirically that reward complexity can be counterproductive, and identify a case where the bottleneck is not the reward function but the observation: no reward configuration induces foraging, while a change to the observation (frame stacking) does.

This work presents a systematic study of reward shaping in a custom grid-based survival environment. The environment features a single agent on a 64×64 grid that must eat food to prevent starvation, avoid enemies that deal damage, and use shelters for protection. The reward function consists of nine weighted components covering event-driven bonuses, continuous penalties, and spatial gradients. We vary these weights across experiments while holding the agent architecture and all hyperparameters fixed, isolating the effect of reward configuration on learned behavior.

The agent is a Rainbow-lite DQN [Hessel et al., 2018] combining Double DQN, a Dueling architecture, prioritized experience replay with 5-step returns, and several stability techniques (LayerNorm, AdamW, Polyak target updates). Training runs on single consumer GPUs, following work by Obando-Ceron and Castro [2021] and Clark et al. [2024] demonstrating that algorithmic choices can substitute for massive compute.

Section 2 reviews the relevant background. Section 3 describes the survival environment. Section 4 details the agent architecture. Section 5 presents the experiments and results. Sections 6 and 7 discuss findings and conclude.

2 Background

2.1 Reinforcement Learning Fundamentals

We formulate the task as a Markov Decision Process (MDP) defined by a tuple (S, A, P, R, γ): state space S, action space A, transition function $P(s' \mid s, a)$, reward function $R(s, a, s')$,

and discount factor $\gamma \in [0, 1)$ [Sutton and Barto, 2018].

The agent interacts with the environment in discrete time steps $t = 0, 1, 2, \ldots$, receiving reward $r_t \in \mathbb{R}$ at each step. The goal is to find a policy $\pi : S \to A$ that maximizes the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$. The action-value function

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s,\ a_t = a] \tag{1}$$

gives the expected return of taking action a in state s and following π thereafter. The optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}\left[r_t + \gamma \max_{a' \in A} Q^*(s_{t+1}, a')\right], \tag{2}$$

where s_{t+1} is the successor state reached after taking a in s.

Q-learning estimates Q^* directly through temporal difference (TD) updates:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right], \tag{3}$$

where α is the learning rate. The bracketed quantity is the TD error δ_t.

2.2 Deep Q-Networks

When the state space is large, tabular Q-learning is infeasible. Deep Q-Networks (DQN) approximate Q^* with a neural network $Q(s, a; \theta)$ parameterized by weights θ [Mnih et al., 2013, 2015].

Two mechanisms stabilize training. First, *experience replay* stores transitions (s, a, r, s') in a buffer and samples mini-batches uniformly, breaking temporal correlations. Second, a *target network* with parameters θ^- provides stable TD targets. The loss for a sampled transition is:

$$L(\theta) = \left[r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right]^2. \tag{4}$$

The target parameters θ^- are periodically copied from θ.

Exploration uses an ε-greedy policy: with probability ε a random action is taken, otherwise the greedy action $\arg\max_a Q(s, a; \theta)$.

2.3 DQN Extensions

Several extensions address specific weaknesses of the basic algorithm.

Double DQN. Standard DQN overestimates action values because the same network both selects and evaluates the maximizing action. Double DQN [van Hasselt et al., 2016] decouples these steps:

$$y = r + \gamma\, Q\Big(s',\ \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\Big). \tag{5}$$

The online network selects the action; the target network evaluates it.

Dueling architecture. The Dueling network [Wang et al., 2016] decomposes the Q-function into a state value $V(s)$ and an action advantage $A(s, a)$:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a'), \tag{6}$$

where $|A|$ is the number of actions. This decomposition allows the network to learn state values without evaluating every action, which helps in states where the action choice matters little.

Prioritized experience replay. Uniform replay treats all transitions equally. Prioritized experience replay (PER) [Schaul et al., 2016] samples transitions proportionally to their TD error magnitude, revisiting surprising transitions more often. The sampling probability of transition i is:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon_p, \tag{7}$$

where α controls prioritization strength and ϵ_p is a small constant ensuring nonzero probability. Non-uniform sampling introduces bias, corrected by importance-sampling (IS) weights:

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta \Big/ \max_j w_j, \tag{8}$$

where N is the number of stored transitions and β is annealed from a starting value to 1 during training, gradually applying the full bias correction.

Multi-step returns. Instead of bootstrapping from the immediate next state, n-step returns [Sutton, 1988] accumulate rewards over n steps:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-). \tag{9}$$

This propagates reward information faster at the cost of increased variance. Moderate values of n (typically 2–5) improve learning speed in practice.

Rainbow. Hessel et al. [2018] combined six DQN extensions into a single agent: Double DQN, Dueling, PER, multi-step returns, distributional RL (C51), and NoisyNet. An ablation study showed that each component contributes, with prioritized replay and multi-step returns providing the largest individual gains.

We adopt four of these six components: Double DQN, Dueling, PER, and 5-step returns. We omit distributional RL due to implementation complexity and NoisyNet because ε-greedy exploration is sufficient at smaller scales [Obando-Ceron and Castro, 2021, Fortunato et al., 2018].

2.4 Training Stability Techniques

Polyak soft target updates. The original DQN copies θ to θ^- every C steps, creating discontinuities in the targets. Polyak averaging [Lillicrap et al., 2016] replaces the hard copy with an exponential moving average, updating the target at every step:

$$\theta^- \leftarrow (1 - \tau)\, \theta^- + \tau\, \theta, \tag{10}$$

with a small τ. This produces smoother target evolution and removes the need to tune C.

AdamW. The Adam optimizer applies L2 regularization to the gradient, which interacts poorly with adaptive learning rates. AdamW [Loshchilov and Hutter, 2019] decouples weight decay from the gradient update, providing effective regularization that helps maintain network plasticity during long training runs.

LayerNorm. Batch normalization is unreliable in RL because the data distribution is non-stationary and effective batch sizes are small. LayerNorm [Ba et al., 2016] normalizes across features within each sample, independent of the batch, making it suitable for the single-sample updates common in RL.

Learning rate scheduling. The 1cycle policy [Smith and Topin, 2019] ramps the learning rate from a low value to a peak over a short warmup phase, then cosine-anneals it back down. This provides implicit regularization and can accelerate convergence compared to a fixed learning rate.

Plasticity. Neural networks in RL can lose the ability to learn over extended training [Lyle et al., 2023, Dohare et al., 2024]. Weight decay and normalization are effective countermeasures. Our combination of AdamW, LayerNorm, and learning rate annealing addresses this concern.

2.5 Reward Shaping

Ng et al. [1999] proved that adding a potential-based shaping reward $F(s, s') = \gamma\,\Phi(s') - \Phi(s)$ to the environment reward preserves the set of optimal policies. This result applies to shaping rewards derived from a fixed potential function of the state.

Practical reward functions often deviate from this framework. Event-driven bonuses (eating food, taking damage) and threshold-based penalties (hunger warnings) are not derived from a state potential. When multiple such components are combined with different weights, the effective optimization objective changes. The agent maximizes the weighted sum, which may not correspond to any single intended behavior. This kind of mismatch between the specified reward and the behavior the designer actually wants is commonly described as reward hacking [Amodei et al., 2016].

In a survival setting, this creates a tension. A high food reward produces aggressive foraging at the cost of safety. A high death penalty produces shelter-seeking at the cost of starvation. A high shelter reward produces passive camping. The interaction between these components, not just their individual magnitudes, determines the learned policy.

This interaction is the central object of study. We fix the agent and vary only the reward weights to isolate their effect on behavior.

3 Environment

3.1 World Design

The environment is a 64×64 bounded grid with discrete time steps. Each episode runs for at most 1000 steps or until the agent dies (health reaches zero). The world contains

four entity types.

Agent. A single agent with two resource bars: health h and hunger u, both with maximum value $h_{max} = u_{max} = 100$. The agent spawns at the grid center with both bars full. Hunger depletes by 0.2 per tick and by an additional 0.5 when moving. When hunger reaches zero, the agent starves and loses 0.5 health per tick. When hunger exceeds 30% of maximum, health regenerates at 0.5 per tick up to the cap.

Enemies. Mobile entities that deal 15 damage on contact (Manhattan distance ≤ 1). Each tick, an enemy moves with probability 0.5. Within a vision range of 3 (Manhattan distance), enemies pursue the agent along the axis of greatest distance. Outside this range, they move randomly. Enemies cannot enter shelter tiles or tiles occupied by another enemy. The world supports up to 8 enemies simultaneously, with roughly a third of this capacity present at episode start. New enemies appear with probability 0.01 per tick.

Food. Static items that restore 20 hunger when consumed by stepping onto their tile. Up to 36 food items exist simultaneously, with half present at episode start. New food appears with probability 0.05 per tick.

Shelters. Static tiles generated at episode start with density 0.02 per cell (approximately 82 shelters per episode). While in a shelter, the agent takes no damage from enemies. Shelters do not protect against starvation.

Each episode uses a different random seed, producing a unique shelter layout and initial entity placement. Enemy and food spawning continues throughout the episode, maintaining population pressure.

3.2 Observation Space

The agent observes through an egocentric 15×15 window (radius 7). The observation consists of three binary spatial channels:

1. Enemy presence
2. Food presence
3. Shelter presence

Cells outside the grid boundary are zero. Three scalar features are appended: $\frac{h}{h_{\max}}$, $\frac{u}{u_{\max}}$, and a binary shelter indicator. The full observation is a 678-dimensional vector (675 spatial + 3 scalar).

3.3 Action Space

The agent has five discrete actions: stay in place, or move one cell up, down, left, or right. Movement is clamped to the grid boundaries.

3.4 Reward Structure

The reward at each step is a weighted sum of up to twelve components:

$$
\begin{aligned}
r_t = {} & w_{\text{food}}\, i_t^{(\text{eat})} + w_{\text{edm}}\, d_t^{(\text{e})} + w_{\text{sdm}}\, d_t^{(\text{s})} + w_{\text{death}}\, i_t^{(\text{die})} \\
& + w_{\text{hunger}}\, i_t^{(\text{low})} + w_{\text{hprop}}(1 - u_t) + w_{\text{tick}} \\
& + w_{\text{fprox}}\, \Delta\phi_t^{(\text{f})} + w_{\text{eprox}}\, \Delta\phi_t^{(\text{e})} + w_{\text{sprox}}\, \Delta\phi_t^{(\text{s})} \\
& + w_{\text{shelter}}\, i_t^{(\text{safe})},
\end{aligned}
\tag{11}
$$

where $w_{\cdot} \in \mathbb{R}$ are tunable scalar weights; $i_t^{(\text{eat})}, i_t^{(\text{die})}, i_t^{(\text{low})}, i_t^{(\text{safe})} \in \{0, 1\}$ are binary indicators for food eaten, death, low hunger, and shelter safety; $d_t^{(\text{e})}$ and $d_t^{(\text{s})}$ are enemy damage and starvation damage; and $u_t \in [0, 1]$ is the hunger ratio.

The three proximity terms use potential-based reward shaping (PBRS) in delta form. For target type $x \in \{\text{f}, \text{e}, \text{s}\}$ (food, enemy, shelter), define the closeness potential

$$
\phi_t^{(x)} = \frac{d_{\max} - d_t^{(x)} + 1}{d_{\max} + 1} \in [0, 1],
\tag{12}
$$

where $d_t^{(x)}$ is the Manhattan distance to the nearest visible entity of type x and $d_{\max} = 2r_{\text{obs}}$. If no entity is visible, $\phi_t^{(x)} = 0$. The delta reward $\Delta\phi_t^{(x)} = \phi_t^{(x)} - \phi_{t-1}^{(x)}$ is positive when the agent moves closer and negative when it moves away, satisfying the PBRS form of Ng et al. [1999]. Unlike absolute proximity, delta proximity does not reward hovering near a target.

Two proximity terms can be conditionally gated on the hunger ratio. When conditional gating is enabled, the food proximity fires only when $u_t < u_{\text{thresh}}$ (hungry) and the shelter proximity fires only when $u_t \geq u_{\text{thresh}}$ (well-fed). This encodes a behavioral mode switch through the reward function: the agent receives a spatial gradient toward food when it needs to eat and toward shelter when it does not.

Table 1 lists all reward components and their baseline weights.

Table 1: Reward components and baseline weights. Components marked "disabled" are set to zero in the baseline and activated in specific experimental configurations.

Component	Condition	Baseline
Food eaten (w_{food})	Agent steps on food tile	+5.0
Enemy damage (w_{edm})	Per unit of damage received	−0.1
Starvation damage (w_{sdm})	Per unit of starvation damage	−1.0
Death (w_{death})	Health reaches zero	−10.0
Low hunger (w_{hunger})	Per tick when $u_t < u_{\text{thresh}}$	disabled
Hunger proportional (w_{hprop})	Continuous, scales with $(1 - u_t)$	−0.3
Food proximity (w_{fprox})	PBRS delta toward nearest food	+0.1
Enemy proximity (w_{eprox})	PBRS delta toward nearest enemy	disabled
Shelter proximity (w_{sprox})	PBRS delta toward nearest shelter	disabled
Shelter safety (w_{shelter})	Agent in shelter	+0.1
Survival tick (w_{tick})	Per step alive	disabled

These components create competing pressures. Collecting food requires movement, which costs hunger and exposes the agent to enemies. Shelters neutralize enemy damage

but provide no food. The hunger-proportional penalty grows every tick, creating urgency to eat that the agent cannot escape through passivity.

A subtle design consideration is the relative magnitude of continuous per-tick rewards versus sparse event rewards. A per-tick reward of +0.1 accumulated over a 700-step episode produces +70, which can dominate a food reward of +5.0. This creates incentives for passive survival: the agent learns to avoid risk rather than seek food. Similarly, a food proximity reward that is too large relative to the food-eaten reward incentivizes hovering near food without eating it, since eating removes the food and terminates the proximity stream. These failure modes motivate the experimental configurations in Section 5.

Relation to potential-based shaping. The delta proximity terms satisfy the PBRS conditions of Ng et al. [1999] and therefore preserve the set of optimal policies under the base reward. The remaining components (event-driven bonuses, threshold penalties, proportional penalties) are not derived from a state potential. When multiple such components are combined with different weights, the effective optimization objective changes. The agent maximizes the weighted sum, which may not correspond to any single intended behavior. This kind of mismatch between the specified reward and the behavior the designer actually wants is commonly described as reward hacking [Amodei et al., 2016].

These weights are the primary independent variable in our experiments.

4 Agent Architecture

4.1 Network

Since this work isolates the effect of reward shaping, the architecture must be adequate for the task but does not need to be optimal. The key requirement is sufficient representational capacity so that the architecture is not the bottleneck. A convolutional encoder is the natural choice because the observation has spatial structure ($3 \times 15 \times 15$ grid). The Dueling head is inherited from the Rainbow-lite recipe. Channel counts and hidden dimensions follow standard values from the literature rather than task-specific tuning. The resulting network has 970,438 trainable parameters for a 678-dimensional input with 5 output actions, providing ample capacity for the relatively simple decision problem. Specific architectural choices (depth, width, kernel sizes) are not expected to materially affect the reward shaping analysis.

The encoder splits the input into a $3 \times 15 \times 15$ spatial grid and 3 scalar features. Three convolutional layers process the spatial grid:

1. $\text{Conv2d}(3 \to 32,\ 3 \times 3,\ \text{stride } 1,\ \text{pad } 1) \to \text{ReLU} \to \text{LayerNorm}$
2. $\text{Conv2d}(32 \to 64,\ 3 \times 3,\ \text{stride } 1,\ \text{pad } 1) \to \text{ReLU} \to \text{LayerNorm}$
3. $\text{Conv2d}(64 \to 64,\ 3 \times 3,\ \text{stride } 2,\ \text{pad } 0) \to \text{ReLU}$

The third layer reduces the spatial dimensions from 15×15 to 7×7, producing a 3136-dimensional feature vector. This is concatenated with the 3 scalar features and projected through a fully connected layer ($3139 \to 256$) with ReLU and LayerNorm.

The Dueling head splits the 256-dimensional representation into two streams. The value stream maps $256 \to 128 \to 1$. The advantage stream maps $256 \to 128 \to 5$. Q-values are computed as $Q(s, a) = V(s) + A(s, a) - \bar{A}(s)$.

Convolutional layers use Kaiming uniform initialization. Fully connected layers use Xavier uniform initialization. The output layers of both Dueling streams use orthogonal

initialization with gain 0.01, producing near-zero initial Q-values and avoiding large initial TD errors.

4.2 Training Algorithm

Algorithm 1 summarizes the training procedure.

Algorithm 1 Rainbow-lite DQN training.

```
Initialize policy network θ, target network θ⁻ ← θ
Initialize prioritized replay buffer D, n-step accumulator
Collect 10,000 transitions with random policy into D
for t = 1, ..., T do
    Select a_t via ε-greedy from Q(·; θ), decay ε
    Execute a_t, observe r_t, s_{t+1}, d_t
    Compute n-step return; commit to D when ready
    Sample batch from D with PER; compute importance-sampling (IS) weights w_i
    y_i = R_i^(n) + (1 − d_i) γ^n Q(s'_i, arg max_a' Q(s'_i, a'; θ); θ⁻)
    L = (1/B) Σ_i w_i · Huber(Q(s_i, a_i; θ), y_i)
    Update θ via AdamW with gradient clipping; step LR scheduler
    θ⁻ ← (1 − τ) θ⁻ + τ θ
    Update priorities in D from new TD errors
end for
```

Here, $d_i \in \{0, 1\}$ indicates whether sampled transition i is terminal, and B denotes the batch size.

Experience collection. At each step, the transition $(s_t, a_t, r_t, s_{t+1}, d_t)$ enters an n-step accumulator. Once $n = 5$ transitions have accumulated, the oldest is committed to the replay buffer with the n-step return (Equation 9), bootstrap state s_{t+n}, and discount factor γ^n. At episode termination (agent death), remaining entries are flushed with shortened returns. On episode truncation (time limit), pending entries are discarded to prevent cross-episode contamination of n-step returns.

The distinction between termination and truncation matters for the bootstrap target. A terminated episode (death) sets $d = 1$, zeroing the bootstrap term. A truncated episode sets $d = 0$, allowing bootstrapping from the final state since the agent was still alive.

Replay buffer. Transitions are stored in pre-allocated numpy arrays. A SumTree data structure supports $O(\log n)$ priority updates and proportional sampling. Stratified sampling divides $[0, P_{\text{total}}]$ into equal segments and draws one sample per segment, reducing variance compared to pure proportional sampling.

Optimization. AdamW with decoupled weight decay (10^{-4}) on convolutional and linear weights; bias and normalization parameters are exempt. Gradients are clipped to norm 1.0. The learning rate follows a 1cycle schedule: 5% warmup from 10^{-5} to 10^{-4}, then cosine annealing to 10^{-6}.

Exploration. ε-greedy with linear decay from 1.0 to 0.01 over the first half of training.

4.3 Hyperparameters

Table 2 lists all hyperparameters. Values follow established defaults from the literature rather than environment-specific tuning. The focus of this work is reward shaping, so all hyperparameters are held constant across experiments.

Table 2: Agent hyperparameters.

Parameter	Value	Reference
Optimization		
Learning rate (peak)	10^{-4}	
LR schedule	OneCycle, 5% warmup, cosine anneal	Smith and Topin [2019]
Weight decay	10^{-4}	Loshchilov and Hutter [2019]
Batch size	128	
Gradient clip norm	1.0	
Replay buffer		
Buffer capacity	500,000	
Min. buffer size (warmup)	10,000	Mnih et al. [2015]
PER α	0.6	Schaul et al. [2016]
PER β_0	0.4	Schaul et al. [2016]
β annealing steps	$T/2$	
n-step returns	5	Hessel et al. [2018]
Target network		
Polyak τ	0.005	Lillicrap et al. [2016]
Exploration		
ε start / end	1.0 / 0.01	Mnih et al. [2015]
ε decay steps	$T/2$	
Network		
CNN channels	(32, 64, 64)	
Merge hidden dim	256	
Dueling head hidden dim	128	
Training		
Discount γ	0.99	
Total timesteps T	varies	
Eval frequency	5,000 steps	
Eval episodes	10	

5 Experiments

5.1 Experimental Setup

Each run trains for $T = 2{,}000{,}000$ timesteps on a single consumer GPU (RTX 2060/3060 class). Greedy evaluation (20 episodes, $\varepsilon = 0$) runs every 5,000 steps during training. Final benchmarks evaluate the best checkpoint over 100 episodes with fixed seeds. We

report four behavioral metrics: mean survival time (ticks), mean food eaten, mean damage taken, and death rate. Each configuration runs with 3 random seeds for variance estimation. The agent architecture and all hyperparameters (Table 2) are held constant across runs; only the reward weights change.

Heuristic baseline. A scripted agent provides a reference for non-learned behavior. The heuristic camps in the nearest shelter and leaves only when the hunger ratio drops below 0.5, at which point it moves toward the nearest food. It flees from enemies within Manhattan distance 5. This agent survives approximately 764 steps on average, eats roughly 1.2 food items per episode, and dies in about 96% of episodes. The remaining 4% reach the 1000-step time limit.

5.2 Failure Modes

Early experiments revealed three degenerate policies that emerged reliably under certain reward configurations.

Oscillation (C3 violation). The agent alternates between two adjacent cells indefinitely. This occurs when a per-tick survival reward provides sufficient return through passivity. With $w_{\text{tick}} = 0.1$ and $\gamma = 0.99$, the discounted return from oscillating for 700 ticks until starvation death is approximately $+9.9$, while the death penalty discounted to $t = 700$ contributes only -0.13. The agent has found the true optimal policy for this reward function: collect ticks while avoiding risk. Constraint C3 prevents this by requiring that passive returns remain negative.

Food hovering (C2 violation). The agent moves near food but does not eat it. This occurs when the food proximity reward exceeds the hunger penalty. An agent 5 cells from food with $w_{\text{prox}} = 0.5$ earns $+0.33$ per tick from proximity, while the average hunger penalty is -0.15 per tick, yielding a positive net reward for inaction. Eating the food would remove the proximity stream. Constraint C2 prevents this by bounding w_{prox} below $|w_{\text{hprop}}|/2$.

Suicide (C1 violation). When the proximity gradient is too weak to guide the agent toward food (C1 violated), the agent cannot learn to forage. Without foraging, every tick accumulates hunger-proportional penalty. A stationary agent surviving 700 ticks until starvation accumulates approximately -135 from w_{hprop} and -100 from w_{sdm}, totaling -245 before the death penalty. The agent discovers that dying quickly is preferable: seeking enemies yields a fast death at total cost ≈ -24 (a few enemy hits plus the death penalty), roughly $10\times$ less than slow starvation. This is a rational response to a reward function where living is more expensive than dying.

All three failure modes are instances of reward hacking [Amodei et al., 2016]: the agent optimizes the reward function as specified, but the resulting behavior diverges from the designer's intent. They motivate the feasibility constraints derived in Section 5.3.

5.3 Reward Configurations

The failure modes above show that poorly chosen reward weights produce degenerate policies. To turn this observation into a principled design tool, we derive seven feasibility

constraints (C1–C7) from the environment mechanics. A reward configuration is *feasible* if all seven hold simultaneously.

Let $D = 2r_{\text{obs}}$ denote the observation diameter. The constraints are:

C1 *Anti-stasis.* One step toward food must yield more proximity reward than the hunger penalty from the movement cost: $w_{\text{prox}} > |w_{\text{hprop}}| \cdot c \cdot (D+1) \,/\, u_{\max}$.

C2 *Anti-hovering.* Maximum proximity reward must not exceed the average hunger penalty: $w_{\text{prox}} < |w_{\text{hprop}}|/2$.

C3 *Anti-passivity.* Total tick reward over a passive episode must not exceed accumulated penalties: $w_{\text{tick}} < (|w_{\text{hprop}}|\Sigma_h + |w_{\text{sdm}}|h_s T_{\text{death}} + |w_{\text{death}}|) \,/\, T_{\text{passive}}$, where Σ_h is the cumulative hunger ratio over a stationary episode.

C4 *Enemy flee.* Damage penalty from one hit must exceed the hunger penalty from one evasive step: $|w_{\text{edm}}| \cdot d_e > |w_{\text{hprop}}| \cdot c \,/\, u_{\max}$.

C5 *Starvation escalation.* Per-tick penalty when starving must exceed the max hunger-proportional penalty: $|w_{\text{sdm}}| \cdot h_s > |w_{\text{hprop}}|$.

C6 *Terminal signal.* Death penalty must exceed the best single event: $|w_{\text{death}}| > w_{\text{food}}$.

C7 *Food trip.* Reaching food at expected distance $\bar{d}$ must be worth the movement cost: $f > \bar{d} \cdot c$.

Each failure mode in Section 5.2 maps to a specific constraint violation: oscillation to C3, food hovering to C2, and suicide to C1.

Table 3 defines three reward configurations. The BASELINE satisfies all constraints. The two diagnostic configurations each violate a specific constraint to confirm the predicted failure mode.

Table 3: Reward configurations. Only modified components are shown; all others use the baseline default (Table 1).

Configuration	Modification	Violated constraint
BASELINE	(none — default weights)	(none)
WEAK_PROXIMITY	$w_{\text{prox}} = 0.02$	C1 (anti-stasis)
NO_DEATH	$w_{\text{death}} = 0$	C6 (terminal signal)
DEATH_ONLY	all weights zero except w_{death}	sparse signal
FREE_MOVEMENT	movement cost $c = 0.0$	diagnostic

Baseline. The default weights (Table 1) satisfy all seven constraints. Zero survival tick eliminates passive reward, strong hunger-proportional penalty (-0.3) creates urgency, and the food proximity gradient ($+0.1$) provides a directional signal without dominating the reward budget. This is the primary configuration expected to produce active foraging.

Weak proximity. Reduces w_{prox} from 0.1 to 0.02, violating C1: the proximity gain from one step toward food (0.02) falls below the hunger penalty from the movement cost ($|w_{\text{hprop}}| \cdot c \cdot (D+1)/u_{\max} = 0.0225$). Without a viable foraging strategy, the agent converges to the suicide failure mode: seeking enemies to die quickly or oscillating to accelerate starvation, both of which truncate the accumulating hunger penalty.

No death. Removes the death penalty ($w_{\text{death}} = 0$), violating C6. All per-step shaping signals remain. Tests whether TD bootstrapping of a terminal signal is necessary

for learning survival, or whether the accumulated per-step penalties (hunger, starvation damage, enemy damage) are sufficient on their own.

Death only. Removes all per-step rewards, leaving only the terminal death penalty ($w_{\text{death}} = -10$). The agent receives zero signal until the episode ends in death. This is the sparsest possible reward for the survival task and tests the credit assignment limit: can the agent attribute a death at $t = 700$ to actions taken hundreds of steps earlier, with no intermediate gradient?

Free movement. Sets the movement hunger cost to zero while keeping all reward weights at baseline values. If the agent still oscillates under this configuration, oscillation is a training artifact rather than a reward-rational strategy, since the reward incentive to move toward food is unambiguously positive.

5.4 Engineered Reward Progression

The diagnostic configurations above confirm that constraint violations produce predictable failures. A natural follow-up question is whether a carefully designed reward function, satisfying all constraints and adding spatial gradients for each behavioral objective, can match or exceed the heuristic. We test a sequence of engineered configurations that progressively refine the reward signal, each iteration addressing a specific limitation identified in the previous one.

Structural incompatibility of hunger-proportional penalty. The baseline reward includes a continuous hunger-proportional penalty $w_{\text{hprop}}(1-u_t) = -0.3(1-u_t)$. This creates a gradient encouraging the agent to maintain high hunger. However, every movement costs $c = 0.5$ hunger, triggering an incremental penalty of $|w_{\text{hprop}}| \cdot c/u_{\max} = 0.0015$ per step. At $\gamma = 0.99$, the present value of this penalty stream over the expected remaining episode is

$$\text{PV}_{\text{move}} = \frac{0.0015}{1 - 0.99} = 0.15, \tag{13}$$

while the maximum food proximity reward from one step is approximately $0.1/15 \approx 0.0067$. The movement penalty exceeds the approach reward by a factor of 22. Under this reward function, not moving is always Q-value optimal regardless of hunger level, producing a stasis equilibrium that no proximity gradient can overcome.

This structural incompatibility motivates removing w_{hprop} and replacing it with a threshold-based penalty that does not couple to movement cost.

Configuration E1: threshold penalty with proximity gradients. The first engineered configuration removes w_{hprop}, introduces a flat low-hunger penalty ($w_{\text{hunger}} = -0.5$ when $u_t < 0.3$), strengthens the food reward ($w_{\text{food}} = 8.0$) and food proximity ($w_{\text{fprox}} = 0.15$), and adds an enemy proximity gradient ($w_{\text{eprox}} = -0.5$). The flat penalty creates urgency without penalizing movement, since the penalty magnitude is independent of hunger level changes.

E1 produces an agent that forages well and avoids enemies: it eats roughly twice as much food as the heuristic (2.0 vs 1.19 per episode) and takes less damage per tick. However, the agent moves every tick, burning hunger approximately 3.5 times faster than

the heuristic. The heuristic, by contrast, idles near shelter when well-fed, conserving hunger for 60–70% of each episode.

Configuration E4: shelter proximity and survival tick. To address the missing shelter-seeking behavior, E4 adds a shelter proximity gradient ($w_{\text{sprox}} = 0.15$) that mirrors the food proximity signal and gates both on the hunger threshold: food proximity fires only when hungry ($u_t < 0.5$), shelter proximity fires only when well-fed ($u_t \geq 0.5$). A small survival tick ($w_{\text{tick}} = 0.05$) directly rewards each step alive. E4 achieves 695 mean survival with nine active reward components, still below the heuristic.

Configuration E5: minimal reward set. E5 strips the reward to three PBRS delta proximities (food, enemy, shelter) plus the death penalty. All event-driven rewards (w_{food}, w_{edm}, w_{sdm}), threshold penalties (w_{hunger}), and binary signals (w_{shelter}, w_{tick}) are set to zero. The hypothesis is that fewer signals reduce interference in Q-value estimation. The three proximity gradients directly encode the behavioral mode switch: hungry agents receive a gradient toward food, well-fed agents toward shelter, and all agents away from enemies. E5 achieves 715 mean survival, the best of any reward-only configuration, despite eating only 0.66 food per episode.

The foraging gap. E5 learns coherent shelter and flee behavior but barely forages: 0.66 food per episode versus the heuristic's 1.19. The survival gap of 54 ticks corresponds to the food deficit: roughly 0.6 fewer food items at approximately 100 ticks of additional life per item.

We tested explicit food rewards (w_{food}) at magnitudes from 0.3 to 5.0. At 5.0, the food reward causes the agent to pursue food through enemy zones; survival drops to 470. At lower magnitudes, the agent forages more but takes more damage; survival does not exceed E5's shelter-camping policy. No tested magnitude produces both increased foraging and maintained enemy avoidance.

5.5 Temporal Context

Frame stacking provides temporal context by concatenating the spatial grids from the last k time steps. The base observation has 3 spatial channels on a 15×15 grid; a 4-frame stack provides 12 channels. An enemy occupying different cells across consecutive frames reveals its direction and speed. The scalar features (health, hunger, shelter indicator) are taken from the current step only.

We train E5 with $k = 4$ frames for 5,000,000 steps. The replay buffer is reduced from 1,000,000 to 250,000 transitions to accommodate the larger observation size. All other hyperparameters follow Table 2 except for slower target updates ($\tau = 0.002$) and extended exploration (ε decay over 1,000,000 steps).

5.6 Results

Table 4 reports benchmark performance over 100 episodes with fixed seeds (1000–1099). Each DQN result uses the best checkpoint across training, selected by evaluation survival.

The frame-stacking agent significantly outperforms the heuristic across all three seeds ($p < 0.05$). It eats 4.12 food items per episode, more than three times the heuristic, while

Table 4: Benchmark results (100 episodes, mean ± 95% CI). DQN results report the best of three seeds.

Configuration	Survival	Food	Damage	Death	vs. Heuristic
Heuristic	768 ± 19	1.19	99.5	95%	—
E5 (minimal)	715 ± 19	0.66	101	100%	-54 ($p < 0.001$)
E5 + food reward (5.0)	470 ± 26	0.81	100	100%	-298 ($p < 0.001$)
E5 + extended training	699 ± 16	0.39	100	100%	-70 ($p < 0.001$)
E5 + frame stacking	$\mathbf{811 \pm 25}$	4.12	98.5	88%	$+\mathbf{43}$ ($p = 0.007$)

taking comparable damage. 12% of episodes reach the 1000-step time limit, something no other DQN configuration achieved.

Extended training (5,000,000 steps on E5 without frame stacking) does not help. All three seeds peak between steps 1.7–2.8 million and decline afterward. Longer training hardens the shelter-camping policy rather than discovering foraging.

5.7 Summary of Observations

Three empirical patterns emerge from the experiments.

Fewer reward components produce better policies. E5 with four reward components outperforms E4 with nine (715 vs 695 mean survival), despite receiving less information per step.

Reward scaling does not resolve the foraging gap. Adding a food reward to E5 at magnitudes from 0.3 to 5.0 either degrades enemy avoidance (high values) or fails to improve survival (low values). Extended training (5M steps) also does not help: all seeds peak early and decline.

Frame stacking enables foraging. The only modification that produces foraging behavior competitive with the heuristic is frame stacking. The reward function is unchanged; only the observation changes. The frame-stacking agent eats 4.12 food per episode (vs 0.66 without), but the foraging policy is transient: survival peaks early in training (856–867 at step 350K) and declines as training continues. The best checkpoint captures this transient behavior.

Delta proximity does not cause hovering. Absolute proximity would reward the agent for being near food, creating the hovering failure mode (Section 5.2). Delta proximity rewards only the change in distance. All engineered configurations use delta proximity, and none exhibit hovering.

6 Discussion

The results raise two questions: why does the agent not learn to forage under reward shaping alone, and why does frame stacking enable it? We propose three hypotheses, each with a suggested falsification test.

Reward design as constrained optimization. The feasibility constraints C1–C7 formalize reward design as a system of inequalities. Satisfying all constraints is necessary but not sufficient: the baseline satisfies all seven yet fails due to the structural incompatibility between hunger-proportional penalty and movement cost. The progression from E4 to E5 shows that removing harmful signals is more effective than adding informative ones. The constraints rule out degenerate policies but do not guarantee the intended behavior.

H1: State aliasing. The single-frame observation maps distinct situations to identical representations. A food tile near an enemy looks the same regardless of whether the enemy is approaching or retreating. The Q-network must assign a single value to both cases, which may average out the foraging benefit. Frame stacking resolves this by making enemy trajectories visible, allowing the network to distinguish safe from dangerous foraging states. *Falsification:* freeze enemies in place (remove the trajectory dimension) and retrain with frame stacking. If frame stacking still outperforms single-frame, the advantage is not primarily from trajectory detection, and this hypothesis is wrong.

H2: Discount blindness. At $\gamma = 0.99$, the death penalty of -10 at step 700 has present value $-10 \times 0.99^{700} \approx -0.009$. The survival benefit of eating one food item (delaying death by roughly 100 steps) is worth approximately 0.006 in present value. This signal may be too small relative to TD estimation error for the network to learn from. *Falsification:* train with $\gamma = 1$ and a small survival tick ($w_{\text{tick}} = 0.01$) to make the survival benefit of eating undiscounted. If foraging emerges without frame stacking, the discount factor is a contributing cause of the foraging gap.

H3: Exploration insufficiency. Under ε-greedy with $\varepsilon = 0.01$ (post-decay), the agent rarely leaves shelter. Few foraging transitions enter the replay buffer, so the network has little data from which to learn foraging value. Frame stacking may help not because of what it represents, but because the larger observation space and different training dynamics produce a different exploration pattern during the critical early phase. *Falsification:* replace ε-greedy with an intrinsic motivation method (e.g., RND) that encourages visiting novel states, and retrain without frame stacking. If foraging emerges, insufficient exploration was the primary bottleneck.

These hypotheses are not mutually exclusive. The foraging gap may result from a combination of state aliasing, weak reward signal, and insufficient exploration, with frame stacking partially addressing all three.

Policy collapse. The frame-stacking agent's foraging policy is transient. It peaks early in training and collapses as the agent converges toward shelter-camping. One interpretation is that the flee gradient (-0.5) eventually dominates the food proximity gradient ($+0.15$) as Q-value estimates sharpen. Another is that the foraging policy is a side effect of early noisy exploration that the agent "unlearns" as it optimizes more precisely. Distinguishing these interpretations requires examining the Q-value landscape at different training stages.

Limitations. The frame-stacking result relies on early checkpoint selection. The environment is a single grid world, and the findings may not generalize to continuous-action domains or tasks with different time scales.

7 Conclusion

We studied how reward function design affects learned behavior in a multi-objective survival task. By varying reward weights while holding the agent fixed, we identified three failure modes and derived seven feasibility constraints that prevent them. A minimal configuration with four components outperforms alternatives with up to nine.

Reward shaping alone does not match a scripted heuristic in our experiments. No tested food reward magnitude closes the foraging gap: large values degrade enemy avoidance, small values fail to change behavior, and extended training reinforces shelter-camping rather than discovering foraging.

Frame stacking, without any change to the reward function, produces an agent that significantly outperforms the heuristic (811 vs 768 mean survival, $p = 0.007$), eats three times more food, and survives to the episode time limit in 12% of episodes. The foraging policy is transient, peaking early in training and collapsing with continued optimization.

Why reward shaping alone fails and why frame stacking succeeds remain open questions. We proposed three hypotheses (state aliasing, discount blindness, exploration insufficiency) with concrete falsification tests. Future work includes running these tests, investigating the mechanism behind policy collapse, and testing whether combining frame stacking with a moderate food reward ($w_{\text{food}} = 0.3$) can stabilize the foraging policy.

References

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016. doi: 10.48550/arXiv.1606.06565.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Jack Clark et al. Beyond the rainbow: High performance deep RL on a desktop PC. *arXiv preprint arXiv:2411.03820*, 2024.

Shibhansh Dohare, Qingfeng Lan, and A. Rupam Mahmood. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.

Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell, and Shane Legg. Noisy networks for exploration. In *Proceedings of the International Conference on Learning Representations*, 2018.

Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2016.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations*, 2019.

Clare Lyle, Mark Rowland, and Will Dabney. Understanding plasticity in neural networks. In *Proceedings of the International Conference on Machine Learning*, 2023.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518 (7540):529–533, 2015.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the International Conference on Machine Learning*, 1999.

Johan S. Obando-Ceron and Pablo Samuel Castro. Revisiting rainbow: Promoting more insightful and inclusive deep RL research. In *NeurIPS Deep RL Workshop*, 2021.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations*, 2016.

Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. *arXiv preprint arXiv:1708.07120*, 2019.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2016.